

-TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO BÀI TẬP LỚN

Đề tài: Xây dựng chatbot hỏi đáp về pháp luật Việt Nam

Lớp : 168482

Học phần : Lưu trữ và xử lý dữ liệu lớn

Mã học phần : IT4931

Giảng viên hướng dẫn : TS. Trần Việt Trung

Danh sách thành viên nhóm 12:

Họ và tên	Mã số sinh viên
Phạm Quang Huy	20225338
Trương Huy Khuê	20225348
Tạ Minh Hiếu	20225317
Hoàng Thành Long	20225207
Nguyễn Cảnh Huy	20225334

MỤC LỤC

Hà Nội, tháng 5 năm 2026

Contents

Contents	1
LỜI NÓI ĐẦU	2
Phân công công việc	3
CHƯƠNG 1: ĐỊNH NGHĨA BÀI TOÁN	4
1.1. Bài toán lựa chọn	5
1.2. Phân tích mức độ phù hợp với dữ liệu lớn	5
1.2.1. Volume.....	5
1.2.2. Velocity.....	5
1.2.3. Veracity.....	6
1.2.4. Variety.....	6
1.2.5. Value	6
1.3. Phạm vi và giới hạn dự án	7
1.3.1. Nằm trong phạm vi triển khai	7
1.3.2. Ngoài phạm vi triển khai.....	7
CHƯƠNG 2: KIẾN TRÚC THIẾT KẾ	8
2.1. Kiến trúc tổng thể	9
2.2. Các thành phần và vai trò	10
2.3. Sơ đồ luồng dữ liệu và tương tác giữa các thành phần	11
CHƯƠNG 3: CHI TIẾT TRIỂN KHAI	13
3.1. Mã nguồn kèm tài liệu	13
3.2. File cấu hình theo môi trường	14
CHƯƠNG 4: BÀI HỌC KINH NGHIỆM	15
4.1. Thu thập dữ liệu và Vượt tường lửa bảo mật Cloudflare	16
4.2. Kiến trúc Lambda Hội tụ và Tối ưu hóa chi phí OpenAI	17
4.3. Đảm bảo tính nhất quán của luồng Xử lý sự kiện (Stream Processing)	18
4.4. Thiết lập kho dữ liệu thô Master Dataset	20
4.5. Tích hợp hệ thống - K8s pitfalls	21
4.7. Giám sát và gỡ lỗi	26
4.8. Mở rộng	27
4.9. Chất lượng dữ liệu và kiểm thử	29
4.10. Bảo mật và quản trị	31
4.11. Chịu lỗi	32

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	36
5.1. Kết quả đạt được.....	36
5.2. Hạn chế.....	37
5.3. Hướng phát triển.....	37
DANH MỤC TÀI LIỆU THAM KHẢO.....	38

LỜI NÓI ĐẦU

Trong bối cảnh hệ thống pháp luật Việt Nam ngày càng phức tạp với hàng trăm nghìn văn bản quy phạm được ban hành, sửa đổi và cập nhật liên tục, nhu cầu tra cứu thông tin pháp lý nhanh chóng và chính xác trở nên cấp thiết hơn bao giờ hết. Tuy nhiên, việc tiếp cận và hiểu đúng các văn bản luật vẫn là rào cản lớn đối với đại đa số người dân và doanh nghiệp.

Xuất phát từ thực tiễn đó, nhóm 12 lựa chọn xây dựng hệ thống chatbot hỏi đáp pháp luật Việt Nam theo mô hình RAG (Retrieval-Augmented Generation) làm đề tài cho học phần Lưu trữ và Xử lý Dữ liệu Lớn. Dự án không chỉ là bài toán kỹ thuật về Big Data mà còn là nỗ lực ứng dụng công nghệ để giải quyết một vấn đề xã hội thực tiễn.

Báo cáo này trình bày toàn bộ quá trình thiết kế, triển khai và đánh giá hệ thống từ pipeline thu thập dữ liệu tự động vượt rào bảo mật Cloudflare, kiến trúc Lambda hội tụ xử lý dữ liệu thời gian thực và theo lô, đến tầng tìm kiếm lai kết hợp BM25 và vector search, toàn bộ vận hành trên hạ tầng Kubernetes. Nhóm cũng dành một phần đáng kể để chia sẻ các bài học thực chiến thu được trong quá trình triển khai, với mong muốn đây là tài liệu tham khảo hữu ích cho các dự án tương tự.

Nhóm xin gửi lời cảm ơn chân thành đến TS. Trần Việt Trung đã tận tình hướng dẫn và đồng hành cùng nhóm trong suốt học kỳ.

Phân công công việc

STT	Họ và tên	MSSV	Công việc
1	Phạm Quang Huy	20225338	Chia việc cho các thành viên trong nhóm; Triển khai Kafka; Triển khai Redis; Triển khai Backend FastAPI - RAG pipeline; Ghép nối các service thành hệ thống hoàn chỉnh
2	Trương Huy Khuê	20225348	Triển khai module Crawler; Triển khai Frontend React; Chạy crawl dữ liệu; Viết test; Làm slide
3	Tạ Minh Hiếu	20225317	Triển khai module Crawler; Triển khai Frontend React; Chạy crawl dữ liệu; Viết test; Làm slide
4	Hoàng Thành Long	20225207	Triển khai module Crawler; Chạy crawl dữ liệu; Viết báo cáo; Triển khai Grafana cho ES; Triển khai MinIO
5	Nguyễn Cảnh Huy	20225334	Triển khai giám sát Prometheus + Grafana; Triển khai lên Kubernetes; Triển khai Elasticsearch; Triển khai Spark (batch + streaming)

CHƯƠNG 1: ĐỊNH NGHĨA BÀI TOÁN

1.1. Bài toán lựa chọn

Dự án tập trung giải quyết bài toán xây dựng hệ thống Hỏi - Đáp pháp luật Việt Nam theo mô hình RAG (Retrieval-Augmented Generation) dựa trên ngôn ngữ tự nhiên. Hệ thống tự động thu thập văn bản pháp luật thực tế từ nguồn thuvienphapluat.vn, tiến hành tiền xử lý, băm nhỏ, lập chỉ mục vector trên quy mô lớn.

Khi người dùng đặt câu hỏi bằng ngôn ngữ tự nhiên tiếng Việt, hệ thống thực hiện truy xuất ngữ cảnh (context retrieval) qua cơ chế tìm kiếm lai (Hybrid Search) và chuyển tiếp dữ liệu đến mô hình ngôn ngữ lớn (LLM - GPT-4o mini) nhằm sinh câu trả lời chính xác, đáng tin cậy và bắt buộc phải kèm theo trích dẫn nguồn.

1.2. Phân tích mức độ phù hợp với dữ liệu lớn

Bài toán xây dựng chatbot pháp luật là một bài toán Dữ liệu lớn điển hình. Sự phù hợp và tính bắt buộc phải áp dụng công nghệ Big Data được minh chứng rõ ràng qua mô hình 5V dưới đây:

1.2.1. Volume

Corpus văn bản pháp luật Việt Nam hiện hành vô cùng đồ sộ với quy mô lên tới hàng trăm nghìn văn bản. Nhiều văn bản như Bộ luật Dân sự, Luật Doanh nghiệp hay Luật Đất đai có độ dài hàng trăm trang với cấu trúc phân cấp sâu. Khi hệ thống tiến hành băm nhỏ văn bản theo ranh giới câu để phục vụ mô hình RAG, Số lượng phân đoạn thực tế ghi nhận trong môi trường thử nghiệm đạt ~10.700 chunks được trích xuất từ 2.070 văn bản pháp luật thuộc 31 danh mục. Mỗi chunk này tiếp tục được chuyển đổi thành một vector embedding độ phân giải cao với không gian 1536 chiều (mô hình text-embedding-3-small). Việc lưu trữ, quản lý và thực hiện tìm kiếm ma trận khoảng cách vector trên hàng triệu vector 1536 chiều vượt xa giới hạn xử lý và dung lượng bộ nhớ của các hệ quản trị cơ sở dữ liệu quan hệ truyền thống, đòi hỏi phải sử dụng các giải pháp lưu trữ và lập chỉ mục phân tán chuyên dụng như Elasticsearch.

1.2.2. Velocity

Hệ thống pháp luật là một thực thể động, biến đổi không ngừng. Trung bình mỗi ngày có hàng chục văn bản mới được ban hành, sửa đổi, bổ sung hoặc hết hiệu lực ở nhiều

cấp quản lý. Yêu cầu đặt ra đối với một chatbot pháp luật là phải có tính cập nhật cực kỳ cao. Nếu một văn bản luật mới ban hành ảnh hưởng đến quyền lợi của doanh nghiệp, chatbot không thể đợi đến một vài tuần hoặc vài tháng sau khi mô hình LLM được huấn luyện lại mới biết đến thông tin đó. Hệ thống yêu cầu phải thiết lập một pipeline xử lý dữ liệu thời gian thực (Real-time data pipeline).

1.2.3. Veracity

Đối với ngành Luật, thông tin sai lệch dù chỉ một từ hay một dấu câu cũng có thể dẫn đến những hậu quả pháp lý nghiêm trọng và tổn thất tài chính cho người sử dụng. Do đó, tính xác thực là yêu cầu tối thượng của hệ thống. Trong bối cảnh các mô hình LLM luôn có tỷ lệ ảo giác nhất định, hệ thống phải thiết lập cơ chế kiểm soát chất lượng nghiêm ngặt: kiểm tra trùng lặp dữ liệu (Deduplication) 4 tầng từ Crawler đến Elasticsearch để tránh sai sót dữ liệu, áp dụng Grounded RAG để ép buộc mô hình sinh câu trả lời bám sát 100% ngữ cảnh thực tế, và xây dựng hệ thống Log kiểm toán để theo dõi, đánh giá độ chính xác của bước truy xuất ngữ cảnh.

1.2.4. Variety

Dữ liệu đầu vào của hệ thống thuộc dạng dữ liệu phi cấu trúc (Unstructured Data) và bán cấu trúc (Semi-structured Data). Nguồn dữ liệu thô được thu thập từ Web dưới dạng các trang HTML hỗn hợp, chứa nhiều thành phần rác (quảng cáo, thanh điều hướng, mã script, bảng biểu phức tạp). Bản thân nội dung văn bản luật cũng sở hữu cấu trúc phân cấp đặc thù nghiêm ngặt (Luật, Chương, Mục, Điều Khoản, Điểm) và đa dạng về chủng loại (Nghị quyết, Thông tư, Nghị định, Quyết định). Hệ thống cần áp dụng các kỹ thuật xử lý ngôn ngữ tự nhiên quy mô lớn và các hàm chuyển đổi tùy biến để chuẩn hóa cấu hình dữ liệu, loại bỏ mã HTML rác, đồng bộ bảng mã Unicode NFC và băm văn bản một cách thông minh mà không làm phá vỡ ngữ cảnh của các điều luật.

1.2.5. Value

Việc xây dựng một hệ thống dữ liệu lớn xử lý văn bản pháp luật mang lại giá trị thực tiễn vô cùng to lớn cho xã hội. Hệ thống giúp đại chúng hóa thông tin pháp luật, cho phép bất kỳ người dân nào cũng có thể tiếp cận và hiểu rõ quyền lợi, nghĩa vụ của mình thông qua một giao diện trò chuyện thân thiện. Đối với các doanh nghiệp, hệ thống là một

công cụ đặc lực giúp kiểm soát rủi ro pháp lý, cập nhật nhanh chóng các chính sách thuế, xuất nhập khẩu và đầu tư, từ đó tối ưu hóa chi phí vận hành và nâng cao năng lực cạnh tranh.

1.3. Phạm vi và giới hạn dự án

1.3.1. Nằm trong phạm vi triển khai

- **Thu thập dữ liệu:** Xây dựng module crawler tự động dựa trên Patchright và BeautifulSoup, hỗ trợ bypass cơ chế chống bot của Cloudflare.
- **Pipeline xử lý:** Triển khai luồng Spark Structured Streaming (Speed Layer) và Spark Batch (Batch Layer) đồng thời xử lý trên dữ liệu lớn.
- **Lưu trữ & Tìm kiếm:** Thiết lập hệ thống lưu trữ đối tượng MinIO (Master Dataset) và cơ sở dữ liệu Elasticsearch 8.x tích hợp cấu hình phân tích tiếng Việt chuyên dụng (analysis-icu). Tìm kiếm lai kết hợp BM25 truyền thống và kNN Vector Search.
- **Giao diện & Điều phối:** Xây dựng API Web bằng FastAPI (Python) tích hợp các cơ chế tối ưu chi phí và ngữ cảnh nâng cao như Smalltalk Prefilter, Query Rewriting, Retrieval, Score Threshold; và giao diện Chat UI bằng React + TypeScript.
- **Quản trị & Giám sát:** Triển khai log kiểm toán (phapluat-audit) tự động ẩn danh hóa PII, hệ thống sao lưu Elasticsearch Snapshot sang MinIO, và bộ công cụ giám sát hạ tầng Prometheus + Grafana + Kibana.
- **Sao lưu và khôi phục:** Thiết lập cơ chế sao lưu ảnh đĩa Elasticsearch Snapshot lưu trữ trực tiếp sang hệ thống lưu trữ đối tượng MinIO.
- **Kiểm thử & Đánh giá:** Thiết lập hệ thống kiểm thử đơn vị tự động với pytest và Framework offline đo đặc chất lượng RAG dựa trên chỉ số precision@k và recall@k.

1.3.2. Ngoài phạm vi triển khai

- Hệ thống không xử lý các dạng câu hỏi mang tính chất tư vấn pháp lý cá nhân chuyên sâu, chỉ phục vụ mục đích tra cứu văn bản quy phạm gốc.
- Không hỗ trợ tính năng đa ngôn ngữ và không phát triển ứng dụng trên thiết bị di động (mobile app).

IT4931 - Lưu trữ và xử lý dữ liệu lớn

- Không thực hiện phân bổ hay deploy hệ thống lên các nền tảng điện toán đám mây công cộng (cloud public)
- Hệ thống không tích hợp các tính năng phục vụ xác thực, đăng nhập tài khoản người dùng

CHƯƠNG 2: KIẾN TRÚC THIẾT KẾ

2.1. Kiến trúc tổng thể

Hệ thống áp dụng mô hình Lambda hội tụ (Unified Lambda Architecture), trong đó Speed Layer và Batch Layer cùng ghi vào một Elasticsearch index duy nhất mang tên phapluat, sử dụng mã định danh chunk xác thực (deterministic chunk_id) để thực hiện upsert an toàn, không tạo bản sao dữ liệu trùng lặp.

Speed Layer (xử lý thời gian thực, 24/7): Crawler đẩy dữ liệu vào Kafka topic van-ban-phap-luat. Deployment kafka-consumer lắng nghe Kafka, ghi từng văn bản thành file JSON riêng biệt vào MinIO tại đường dẫn phapluat/master/{YYYY}/{MM}/{doc_id}.json (master dataset). Deployment spark-streaming đọc luồng từ Kafka, thực hiện làm sạch, băm nhỏ văn bản, tạo embedding vector, và ghi kết quả lên Elasticsearch.

Batch Layer (xử lý theo lịch định kỳ): K8s CronJob spark-batch (0 */6 * * *) đọc toàn bộ master dataset từ MinIO master/ qua thư viện boto3, thực hiện dedup-skip (BATCH_FORCE=false) - chỉ xử lý các doc_id chưa tồn tại trong Elasticsearch, giúp tiết kiệm khoảng 99% chi phí gọi API OpenAI. Khi cần làm mới toàn bộ chỉ mục, quản trị viên kích hoạt cờ BATCH_FORCE=true thông qua kubectl apply -f k8s/app/spark-batch-force.yaml.

Serving Layer: Elasticsearch cung cấp tìm kiếm lai (BM25 full-text + kNN HNSW vector, 1536 chiều cosine). Redis đảm nhận vai trò bộ nhớ đệm phản hồi với TTL 3600 giây và cơ chế xóa cache chủ động theo sự kiện. FastAPI điều phối toàn bộ pipeline RAG, bao gồm bộ lọc smalltalk, ngưỡng điểm truy xuất, và viết lại truy vấn. GPT-4o mini (OpenAI) sinh câu trả lời kèm trích dẫn nguồn. Giao diện người dùng được xây dựng bằng React + TypeScript.

Observability & Governance Layer: Prometheus + Grafana giám sát hạ tầng kỹ thuật; Kibana phân tích dữ liệu nghiệp vụ qua audit log. Audit log (index phapluat-audit) tự động ẩn danh hóa PII. ES snapshot → MinIO (CronJob daily) đảm bảo khả năng phục hồi sau thảm họa.

Speed Layer:

- Luồng dữ liệu vận hành: xem mô tả chi tiết trong phần kiến trúc tổng thể trên.

2.2. Các thành phần và vai trò

Hệ thống bao gồm các thành phần chính sau đây, mỗi thành phần đảm nhiệm một vai trò độc lập và phối hợp chặt chẽ với nhau trong toàn bộ pipeline xử lý dữ liệu lớn:

Crawler (Patchright + BeautifulSoup): Thu thập văn bản pháp luật từ thuvienphapluat.vn, vượt hàng rào bảo mật Cloudflare bằng công nghệ Patchright (Playwright drop-in có stealth patches), chạy ở chế độ headed (persistent Chrome profile) là chính để duy trì cookie phiên làm việc; tự động fallback sang headless=True khi môi trường không hỗ trợ giao diện đồ họa. Tự động phát hiện danh mục mới, hỗ trợ cờ --stop-on-seen để dừng khi gặp bài đã thu thập, cài đặt độ trễ ngẫu nhiên 1-2 giây chống rate-limit.

Apache Kafka: Hàng đợi thông điệp trung gian nhận văn bản từ crawler và phân phối tới spark-streaming và kafka-consumer. Sử dụng topic van-ban-phap-luat với message key là article ID để hỗ trợ tracing và consumer dedup. Hỗ trợ replay trong vòng 7 ngày khi cần phục hồi dữ liệu.

MinIO (Object Storage): Hệ thống lưu trữ đối tượng tương thích S3, là nguồn chân lý duy nhất (master dataset). Lưu trữ theo cấu trúc: `phapluat/master/{YYYY}/{MM}/{doc_id}.json` (per-doc JSON bất biến do kafka-consumer ghi), `phapluat/raw/*.csv` (dữ liệu nạp thủ công), và bucket `es-snapshots/` (Elasticsearch snapshots phục hồi sau thảm họa). Batch layer đọc master/ qua boto3 thuần mà không cần hadoop-aws, giảm độ phức tạp runtime.

Spark Structured Streaming (Speed Layer): Xử lý văn bản mới theo thời gian thực 24/7 từ Kafka. Pipeline gồm 5 bước: `readStream Kafka` → `clean_text UDF` (xóa HTML, chuẩn hóa NFC Unicode tiếng Việt) → `chunk_text UDF` (băm nhỏ theo ranh giới câu, tối đa 1000 ký tự, overlap 100) → `foreachBatch` (dedup với ES, batch embedding 100 đoạn/lần gọi API, bulk write ES, xóa Redis cache). Checkpoint lưu tại MinIO `checkpoints/streaming_python/` đảm bảo khả năng khôi phục offset.

Spark Batch - PySpark (Batch Layer): Xử lý lại toàn bộ corpus theo lịch mỗi 6 giờ (CronJob spark-batch). Mặc định chạy dedup-skip, chỉ xử lý các doc_id chưa có trong ES (fetch_existing_doc_ids aggregation). Khi cần tái tạo toàn bộ chỉ mục (upgrade embedding model, sửa chunker), kích hoạt BATCH_FORCE=true qua one-shot K8s Job.

Elasticsearch 8.x: Cơ sở dữ liệu tìm kiếm lai, lưu trữ hai index: phapluat (corpus chính, kết hợp full-text BM25 với kNN HNSW vector 1536 chiều cosine) và phapluat-audit (log kiểm toán đã ẩn danh hóa PII). Sử dụng analyzer tiếng Việt ICU (icu_tokenizer + filter chain: icu_normalizer → icu_folding → lowercase) để chuẩn hóa Unicode NFC và xử lý dấu tiếng Việt. Document ID dạng {doc_id}_{md5(doc_id:chunk_text)[:12]} đảm bảo mỗi chunk có ID xác thực duy nhất, cho phép upsert an toàn.

Redis: Bộ nhớ đệm phản hồi in-memory, khóa băm SHA-256 dựa trên nội dung câu hỏi (cache: + sha256(cau_hoi)), TTL 3600 giây. Cơ chế xóa cache chủ động theo sự kiện (event-driven invalidation) tự động kích hoạt sau mỗi chu kỳ Batch Layer hoàn tất, đảm bảo người dùng không nhận thông tin lỗi thời. Thuật toán giải phóng bộ nhớ LRU khi đầy.

FastAPI Backend: Điều phối toàn bộ pipeline RAG bất đồng bộ. Bao gồm: bộ lọc smalltalk (regex prefilter 5 intent bỏ qua GPT cho greeting/identity/capabilities/thanks/goodbye), ngưỡng điểm truy xuất (từ chối trước GPT khi điểm ES thấp hơn min_retrieval_score), viết lại truy vấn (GPT-4o-mini reformulate câu hỏi tiếp theo dùng lịch sử), dịch vụ audit log có PII sanitization.

Prometheus + Grafana + Kibana: Tầng quan sát phân tách rõ vai trò: Prometheus thu thập chỉ số qua annotation tự động phát hiện dịch vụ; Grafana hiển thị dashboard hạ tầng kỹ thuật (7 panel: request rate, p95 latency, cache hit rate, smalltalk share, refusal rate, avg sources); Kibana phân tích dữ liệu nghiệp vụ (query volume by hour, top categories, latency P95, PII redaction count).

Kubernetes (Minikube): Toàn bộ 15+ service chạy trong namespace law-chatbot trên môi trường Minikube cục bộ. Resource requests/limits được khai báo nghiêm ngặt cho mọi container.

2.3. Sơ đồ luồng dữ liệu và tương tác giữa các thành phần

Luồng dữ liệu của hệ thống được tổ chức theo hai hướng hội tụ về một điểm duy nhất là Elasticsearch index phapluat:

Luồng nạp dữ liệu (Ingest Path): Crawler (mỗi giờ, Patchright) → kiểm tra trùng doc_id với ES + cờ --stop-on-seen → publish lên Kafka topic van-ban-phap-luat. Tại đây luồng chia đôi: (1) kafka-consumer ghi per-doc JSON xuống MinIO master/ (đây là master dataset bất biến); (2) spark-streaming đọc Kafka, xử lý clean → chunk → embed → bulk upsert ES và xóa Redis cache. Song song, spark-batch CronJob (mỗi 6 giờ) đọc MinIO master/ qua boto3, thực hiện dedup-skip, rồi cũng upsert về ES. Điểm hội tụ (convergence point): cả hai luồng speed và batch đều ghi cùng ES index phapluat bằng chunk_id xác thực, đảm bảo upsert an toàn không tạo bản sao.

Luồng phục vụ người dùng (Query Path): Người dùng gửi câu hỏi qua React UI → FastAPI /api/chat kiểm tra Redis cache (hit: trả về <500ms). Nếu cache miss: lọc smalltalk intent → viết lại truy vấn (nếu có lịch sử) → tạo embedding câu hỏi (text-embedding-3-small) → hybrid search ES (kNN boost=0.3 + chunk_text boost=5.0 + title boost=3.0, top_k=10) → kiểm tra ngưỡng điểm → GPT-4o mini sinh câu trả lời với trích dẫn nguồn → ghi Redis cache → ghi audit log (PII sanitized) → trả về người dùng (5-8 giây).

Luồng chống trùng lặp 4 tầng (Deduplication): Tầng 1 - Crawler: kiểm tra ES doc_id trước khi crawl + --stop-on-seen. Tầng 2 - Kafka: message key = article ID. Tầng 3 - Spark: fetch_existing_doc_ids() aggregation trước khi embed (tiết kiệm OpenAI token). Tầng 4 - Elasticsearch: _id xác thực = doc_id + md5(doc_id:chunk_text)[:12], upsert ghi đè bản sao trùng.

Luồng phục hồi sau thảm họa (Disaster Recovery): CronJob es-snapshot (0 2 * * * daily) → scripts/snapshot_elasticsearch.py → ES repository-s3 (keystore pattern) → MinIO bucket es-snapshots/. Khôi phục: POST _snapshot/phapluat-snapshots/snap-XXX/_restore. Backup 3 tầng: ES snapshot (khôi phục nhanh) + MinIO master/ JSON (replay từ đầu qua BATCH_FORCE=true) + CSV backup (phương án cuối).

CHƯƠNG 3: CHI TIẾT TRIỂN KHAI

3.1. Mã nguồn kèm tài liệu

Toàn bộ mã nguồn hệ thống được tổ chức theo cấu trúc module hóa rõ ràng, mỗi thư mục tương ứng với một thành phần kiến trúc độc lập:

crawler/: Module thu thập dữ liệu sử dụng Patchright + BeautifulSoup. Hỗ trợ chế độ --realtime (chỉ publish Kafka, không upload CSV tránh duplicate) và chế độ --url (crawl URL cụ thể cho demo speed layer). Tích hợp cơ chế tự động phát hiện danh mục, dedup trước với ES, và random delay 1-2 giây.

kafka/: Gồm ba thành phần: producer.py (đẩy message lên Kafka), consumer.py (lắng nghe Kafka, ghi per-doc JSON xuống MinIO master/), minio_ingest.py (poll raw/CSV trong MinIO → push Kafka → move sang raw/processed/).

spark/: Tổ chức thành ba thư mục con: streaming/ (speed layer - spark-streaming Deployment, Kafka readStream, foreachBatch), batch/ (batch layer - pipeline.py đọc MinIO master/ qua boto3, schema RAW_DOC_SCHEMA, dedup, embed, upsert ES), utils/ (text.py chứa hàm Python thuần clean_text và chunk_text độc lập với Spark; udfs.py là wrapper UDF của Spark). Việc tách text.py khỏi udfs.py giúp 16 unit tests chạy không cần SparkSession.

backend/: FastAPI application bao gồm endpoint /api/chat (RAG pipeline hoàn chỉnh), services/audit.py (PII sanitization với 4 regex pattern theo thứ tự EMAIL → PHONE → TAX_ID → CMND/CCCD — trong đó CMND và CCCD dùng chung một regex `\b\d{9}\b|\b\d{12}\b`, ghi phapluat-audit), services/cache.py (Redis với SHA-256 key + event-driven invalidation), services/search.py (hybrid search Elasticsearch). Smalltalk router regex prefilter 5 intent. Query rewriting dùng lịch sử 6 lượt gần nhất.

frontend/: Giao diện React + TypeScript (Vite build). Hiển thị câu trả lời kèm trích dẫn doc_id và URL gốc. Gửi lịch sử 6 lượt gần nhất lên backend để hỗ trợ query rewriting cho câu hỏi tiếp theo trong cùng phiên.

scripts/: init_elasticsearch.py (tạo index phapluat với mapping HNSW + Vietnamese ICU analyzer, và index phapluat-audit), snapshot_elasticsearch.py (tạo/liệt kê/phục hồi ES snapshot), bulk_load_to_minio.py (nạp CSV lên MinIO raw/), trigger_batch.py (kích hoạt batch thủ công), demo_speed_1_delete.py + demo_speed_2_recrawl.py (demo end-to-end speed layer).

tests/ và eval/: tests/ chứa 16 unit tests pytest: test_chunker.py (kiểm tra clean_text và chunk_text), test_audit.py (kiểm tra regex PII). Toàn bộ chạy trong ~0.1 giây không cần SparkSession. eval/ chứa gold_set.json (12 truy vấn tiếng Việt mẫu + expected keywords), run_eval.py hỗ trợ hai chế độ --backend (full RAG, có LLM cost) và --direct (retrieval-only, không LLM), đo precision@k, recall@k, category hit rate, mean latency.

k8s/: k8s/infrastructure/ chứa ConfigMap, Secret, Deployment cho hạ tầng (Kafka, ES, Redis, MinIO, Prometheus, Grafana, Kibana). k8s/app/ chứa manifest cho backend, frontend, spark-streaming, kafka-consumer, kafka-worker, spark-batch CronJob, spark-batch-force Job. k8s/deploy.sh build images, apply manifests, wait ready. k8s/start.sh cho local end-to-end với port-forwards.

3.2. File cấu hình theo môi trường

Hệ thống triển khai trên môi trường Kubernetes (Minikube) với cấu hình tách biệt hoàn toàn theo từng thành phần:

Môi trường Kubernetes (Minikube): k8s/infrastructure/ lưu ConfigMap cho cấu hình từng service (Elasticsearch, Kafka, Prometheus scrape config, Grafana datasource provisioning) và Secret cho thông tin nhạy cảm (backend-secret chứa OPENAI_API_KEY; minio-creds chứa access_key và secret_key dùng cho ES keystore initContainer). k8s/app/ lưu Deployment manifests cho backend, frontend, spark-streaming, kafka-consumer, spark-batch CronJob (0 */6 * * *), spark-batch-force Job (one-shot với BATCH_FORCE=true), es-snapshot CronJob (0 2 * * *). Mọi biến nhạy cảm được mount qua envFrom: secretRef, không hard-code trong source. Lệnh triển khai một bước: bash k8s/deploy.sh (build images, apply manifests, wait ready); bash k8s/start.sh cho local end-to-end kèm port-forwards.

Chiến lược CI/CD và giám sát: CI: pytest tests/ (16 unit tests, ~0.1 giây) kiểm tra chunker và PII audit. CD: bash k8s/deploy.sh. Monitoring: Prometheus scrape pods/services có annotation prometheus.io/scrape: “true”, retention 15 ngày, PVC 10Gi. Grafana datasource provisioned tự động (admin/admin123). Logging: stdout → kubectl logs, không cần log aggregator riêng. Data analytics: Kibana dashboard “Legal Chatbot - Document Analytics” import tự động qua kibana-init Job từ file kibana/dashboard-export.ndjson. Force batch re-embed: ./run_batch.sh --force hoặc kubectl apply -f k8s/app/spark-batch-force.yaml.

CHƯƠNG 4: BÀI HỌC KINH NGHIỆM

4.1. Thu thập dữ liệu và Vượt tường lửa bảo mật Cloudflare

Mô tả vấn đề:

- **Bối cảnh và nền tảng:** Tiến hành xây dựng module crawler thu thập văn bản pháp luật lớn từ nguồn trang web.
- **Thách thức gặp phải:** Website áp dụng cơ chế chặn tự động anti-bot challenge của Cloudflare và sở hữu cấu trúc giao diện heavy JS.
- **Tác động đến hệ thống:** Các công cụ cào dữ liệu thông thường đều bị hệ thống bảo mật Cloudflare gắn cờ ngăn chặn, làm nghẽn luồng dữ liệu nạp đầu vào.

Cách tiếp cận đã thử:

- **Cách 1:** Sử dụng cấu hình Scrapy kết hợp thư viện Playwright cổ điển vận hành ở chế độ ẩn danh (headless: True).
Kết quả: Trình duyệt bị Cloudflare phát hiện cấu trúc tự động hóa và chặn (block) truy cập ngay từ cửa ngõ.
- **Cách 2:** Sử dụng Playwright thông thường bổ sung thêm plugin che giấu stealth.
Kết quả: Thông tin cookie phiên làm việc không được lưu vết liên tục, dẫn đến quá trình phân trang bị ngắt quãng nửa chừng.

Giải pháp cuối cùng: sử dụng bộ mã nguồn crawler kết hợp công nghệ Patchright và thư viện BeautifulSoup để bóc tách dữ liệu:

- **Triển khai chi tiết:** Tiến trình chạy trình duyệt ở chế độ hiển thị giao diện (headed mode), chỉ định kênh phân phối Google Chrome channel thực tế (real Chrome channel), sử dụng cấu hình thư mục hồ sơ người dùng liên tục (persistent profile) để duy trì trạng thái cookie an toàn. Mã nguồn crawler tích hợp cơ chế tự động phát hiện tất cả danh mục mới (auto-discover all categories) mỗi khi kích hoạt, tăng số lượng trang quét lên 1-3 trang đầu thay vì 1-2 trang, bổ sung cờ lệnh --stop-on-seen để dừng tiến

trình lập tức khi gặp bài viết cũ và thiết lập khoảng trễ ngẫu nhiên (random delay 1-2 giây) nhằm chống bị khóa tài nguyên (anti-rate-limit).

- Chỉ số và kết quả: Vượt tường lửa bảo mật Cloudflare thành công, nạp trơn tru tập dữ liệu văn bản pháp luật vào cụm lưu trữ.

Điều rút ra: Các giải pháp bypass bot hiện đại không thể chỉ dựa vào việc thay đổi User-Agent, mà bắt buộc phải can thiệp làm sạch các biến môi trường trình duyệt (stealth patches) và duy trì cấu hình persistent session cookie liên tục để bảo toàn trạng thái.

4.2. Kiến trúc Lambda Hội tụ và Tối ưu hóa chi phí OpenAI

Mô tả vấn đề:

- **Bối cảnh và nền tảng:** Triển khai tầng lưu trữ và tìm kiếm lai RAG (Hybrid Search) kết hợp mô hình ngôn ngữ lớn GPT-4o mini trên dữ liệu lớn văn bản pháp luật
- **Thách thức gặp phải:** Kiến trúc Lambda kinh điển yêu cầu phân tách rạch ròi thành hai index độc lập (batch-view và speed-view), gây ra hiện tượng lệch pha dữ liệu lớn (data skew) và đòi hỏi tài nguyên tính toán lớn để gộp kết quả. Đồng thời, việc gọi API sinh vector embedding cho từng dòng dữ liệu đơn lẻ (per-row UDF) làm gia tăng HTTP overhead mạng và chi phí tài chính OpenAI lớn.
- **Tác động đến hệ thống:** Làm chậm tốc độ tính toán của cụm Spark Worker, dễ chạm ngưỡng rate limit hệ thống và gia tăng chi phí Token một cách lãng phí.

Cách tiếp cận đã thử:

- **Cách 1:** Triển khai hàm UDF của Spark gọi API OpenAI sinh vector embedding tuần tự theo từng dòng dữ liệu đơn lẻ.
Kết quả: Thời gian xử lý bị kéo dài nghiêm trọng do nghẽn băng thông HTTP overhead mạng quá cao.
- **Cách 2:** Ép hệ thống sinh lại vector toàn diện (re-embed) cho toàn bộ corpus dữ liệu cũ sau mỗi chu kỳ chạy lô của Batch layer.

Kết quả: Chi phí tài chính tăng tiến theo cấp số nhân dựa trên dung lượng dữ liệu nạp, gây lãng phí lớn.

Giải pháp cuối cùng: Áp dụng kiến trúc Lambda hội tụ (unified Lambda) kết hợp kỹ thuật driver-side batch embedding:

- Triển khai chi tiết: Cấu hình cho cả hai luồng speed layer và batch layer cùng đồng quy ghi dữ liệu về một Elasticsearch index duy nhất mang tên phapluat, sử dụng mã định danh độc nhất cố định từ nội dung băm (chunk_id) để biến mọi hành vi nạp trùng thành lệnh cập nhật ghi đè (upsert). Ở cả hai luồng xử lý, Spark thực hiện chạy hàm fetch_existing_doc_ids() gửi truy vấn aggregation lên Elasticsearch trước để áp dụng chế độ mặc định dedup-skip mode (BATCH_FORCE=false), tự động bỏ qua không xử lý và không gọi Token cho các văn bản đã tồn tại. Chuyển đổi toàn bộ luồng tạo vector sang kỹ thuật driver-side batch embedding với cấu hình đóng gói gom cụm 100 đoạn văn bản trong một lượt gọi API duy nhất (batch_embed() 100 texts/call). Khi có yêu cầu làm mới toàn diện chỉ mục, quản trị viên có thể kích hoạt cờ lực lượng BATCH_FORCE=true thông qua một K8s one-shot Job (spark-batch-force.yaml).
- Chỉ số và kết quả: Loại bỏ hoàn toàn HTTP overhead, tiết kiệm thành công khoảng 99% chi phí gọi API OpenAI cost cho các chu kỳ quét lô sau.

Điểm rút ra: Kiến trúc Lambda hội tụ giúp đơn giản hóa Serving Layer nhờ cơ chế upsert dựa trên mã ID xác thực. Việc lọc trùng dữ liệu lớn trước khi gọi dịch vụ ngoại vi giúp tiết kiệm chi phí token OpenAI và triệt tiêu HTTP overhead.

4.3. Đảm bảo tính nhất quán của luồng Xử lý sự kiện (Stream Processing)

Mô tả vấn đề:

- **Bối cảnh và nền tảng:** Vận hành luồng Lớp tốc độ (Speed Layer) bằng công nghệ Spark Structured Streaming chạy liên tục 24/7 để cập nhật tức thì dữ liệu văn bản mới từ Kafka.

- **Thách thức gặp phải:** Đảm bảo hệ thống đạt khả năng chịu lỗi cao (fault tolerance), dữ liệu không bị lặp bản ghi hoặc không bị rách dữ liệu khi các cụm container tính toán phân tán đột ngột gặp sự cố tắt nghẽn hoặc sập nguồn.
- **Tác động đến hệ thống:** Nếu không lưu vết trạng thái, hệ thống dễ rơi vào tình trạng lệch phân vùng offset, làm mất mát thông tin hoặc làm sai lệch kết quả phản hồi của chatbot.

Cách tiếp cận đã thử:

- **Cách 1:** Triển khai luồng tiêu thụ dòng sự kiện từ Kafka ở chế độ không lưu vết trạng thái phân vùng.
Kết quả: Nguy cơ mất mát thông tin rất cao và mất mốc dữ liệu khi Spark Structured Streaming bị sập.
- **Cách 2:** Áp dụng các mô hình transaction phân tán phức tạp để đồng bộ toàn cụm.
Kết quả: Gây ra độ trễ (latency) hệ thống lớn, làm mất đi tính chất xử lý thời gian thực tức thời của lớp tốc độ.

Giải pháp cuối cùng: Triển khai giải pháp "**Exactly-once**":

- Triển khai chi tiết: Kích hoạt thuộc tính cấu hình lưu vết checkpoint nội bộ của Spark Structured Streaming tại đường dẫn hệ thống tệp tin phân tán checkpoints/streaming_python/ lưu trữ trên MinIO để bảo toàn mốc vị trí offset của Kafka topic van-ban-phap-luat. Đồng thời, phối hợp với tính chất ghi đè an toàn (idempotent target) của Elasticsearch index chung dựa trên mã định danh deterministic chunk_id được tính toán từ nội dung băm của khối chữ.
- Chỉ số và kết quả: Hệ thống đạt trạng thái vận hành ổn định, loại bỏ hoàn toàn rủi ro ghi trùng bản ghi dữ liệu lớn kể cả khi cụm bị restart hoặc chạy replay luồng tin cũ, vòng lặp xử lý end-to-end round-trip đạt hiệu năng ổn định.

Điểm rút ra: Việc triển khai xử lý exactly-once thực tế được cấu thành từ sự kết hợp giữa luồng đẩy ít nhất một lần (At-least-once delivery) và một đích đến có khả năng ghi đè an toàn không đổi trạng thái (Idempotent target).

4.4. Thiết lập kho dữ liệu thô Master Dataset

Mô tả vấn đề:

- **Bối cảnh và nền tảng:** Kiến trúc Lambda yêu cầu duy trì một kho lưu trữ dữ liệu gốc sạch được tinh lọc làm nguồn duy nhất (Master Dataset) phục vụ cho lớp Batch Layer quét tái cấu trúc lại dữ liệu.
- **Thách thức gặp phải:** Kho lưu trữ phải đảm bảo tính nhất quán tuyệt đối về cấu trúc định dạng dữ liệu chữ giữa môi trường phát triển cục bộ và môi trường vận hành cluster. Hệ thống phải đảm bảo khả năng mở rộng linh hoạt khi quy mô dữ liệu corpus tăng trưởng, đồng thời giải quyết bài toán khôi phục sau thảm họa: nếu chỉ mục cơ sở dữ liệu Elasticsearch bị hỏng thì hệ thống phải có nguồn tri thức gốc để tiến hành rebuild toàn diện.
- **Tác động đến hệ thống:** Thiết kế ban đầu sử dụng các file tệp phẳng CSV lớn dùng chung bị lỗi xung đột môi trường tệp phẳng, gây lệch ký tự tiếng Việt và không tối ưu cho việc đọc trích xuất song song dữ liệu lớn.

Cách tiếp cận đã thử:

- **Cách 1:** Sử dụng các file tệp phẳng CSV cục bộ lưu trong thư mục crawler/output/.
Kết quả: Không đảm bảo tính duy trì vĩnh viễn (non-persistent) và không đồng bộ được đa môi trường.
- **Cách 2:** Lưu trữ tập trung dữ liệu thô thu thập được vào các file CSV lớn gộp chung chia sẻ qua phân vùng đĩa hoặc bucket raw/ trên MinIO.
Kết quả: Định dạng monolithic CSV không được tối ưu cho các tác vụ phát lại hoặc nạp lại độc lập từng văn bản (per-doc replay).

Giải pháp cuối cùng: Chuẩn hóa kho Master Dataset về mô hình Per-document JSON phân cấp.

- Triển khai chi tiết: Hệ thống triển khai một cấu phần độc lập mang tên kafka-consumer Deployment chạy thường trực trong cluster, nhận nhiệm vụ lắng nghe Kafka để bóc tách và ghi trực tiếp dòng sự kiện thô thành từng file JSON đơn lẻ lưu trữ theo cấu trúc phân cấp thời gian: `phapluat/master/{YYYY}/{MM}/{doc_id}.json`. Khi luồng batch hoạt động, mã PySpark sử dụng trực tiếp thư viện gọn nhẹ ngôn ngữ thuần boto3 để kết nối list danh mục đối tượng tệp tin theo cấu trúc phân vùng năm/tháng `master/....json`, bỏ qua hoàn toàn lớp cấu hình S3A công kênh.
- Chỉ số và kết quả: Loại bỏ hoàn toàn sự sai lệch định dạng chữ giữa các môi trường, tối ưu hóa tài nguyên quét mục lục nhờ cơ chế chia nhỏ phân vùng theo year/month.

Điểm rút ra: Việc tổ chức Master Dataset theo mô hình per-document JSON bất biến lưu trữ độc lập trên Object Storage giúp bảo toàn nguyên vẹn tính chân lý của dữ liệu. Sử dụng thư viện ngôn ngữ thuần (boto3) giúp tối giản hóa kiến trúc runtime của cụm Spark, loại bỏ sự lệ thuộc vào các gói dependencies của Hadoop.

4.5. Tích hợp hệ thống - K8s pitfalls

Mô tả vấn đề:

- **Bối cảnh và nền tảng:** Dự án tiến hành đóng gói toàn bộ mã nguồn ứng dụng và rất nhiều thành phần hạ tầng dịch vụ lớn để triển khai chạy trong một hạ tầng namespace duy nhất mang tên law-chatbot trên cụm Kubernetes
- **Thách thức gặp phải:** Hệ thống sở hữu hơn 15 dịch vụ đòi hỏi các cơ chế phát hiện dịch vụ, thiết lập thử lại, ngắt mạch an toàn và quản lý sự phụ thuộc tuần tự khi khởi động. Song song với đó, dịch vụ gọi OpenAI API có nguy cơ bị giới hạn tần suất. Đặc biệt, các hành vi mặc định của Kubernetes xung đột nghiêm trọng với kỳ vọng của ứng dụng khiến nhiều bug phức tạp chỉ thực sự bộc lộ ra khi tiến hành deploy thực tế trên môi trường cluster.

- **Tác động đến hệ thống:** Sự xung đột hạ tầng này dẫn đến hiện tượng các Pod liên tục rơi vào trạng thái lỗi vòng lặp, treo tiến trình kiểm tra probe hoặc làm ngắt mạch im lặng các hàm hỗ trợ mà không đẩy ra mã lỗi error log cụ thể.

Cách tiếp cận đã thử và bug gặp phải:

- AFKA_PORT=tcp://... (KAFKA_PORT collision): Theo hành vi mặc định, Kubernetes tự động tiêm (auto-injects) các biến môi trường dạng liên kết dịch vụ có cấu trúc <SVC_NAME>_PORT=tcp://<ip>:<port> cho mọi Service nằm trong cùng một namespace. Container Confluent Kafka đọc trùng biến \$KAFKA_PORT này nhưng bị sai định dạng chuỗi URL thay vì kiểu cấu hình số nguyên (Integer) cổng mạng.
Kết quả: Container crash và văng lỗi exit code 1 ngay sau dòng thông báo "port is deprecated".
Fix: enableServiceLinks: false
- Lỗi Spark NumberFormatException tcp://10.96....:7077: Gặp phải lỗi tương tự khi cấu phần Spark đọc biến môi trường \$SPARK_MASTER_PORT bị K8s inject đè chuỗi URL.
Kết quả: Làm sập Pod container.
Fix: Thêm thuộc tính cấu hình nghiêm ngặt cờ thuộc tính enableServiceLinks: false vào tất cả các tệp manifest deployment của Spark, Kafka và backend để triệt tiêu hoàn toàn hành vi tự động tiêm URL liên kết rác của Kubelet.
- Lỗi Spark Master bind địa chỉ mạng thất bại (Cannot assign requested address): Thiết lập tham số địa chỉ mạng SPARK_MASTER_HOST=spark-master bị hệ thống DNS phân giải thành địa chỉ IP ClusterIP của Service, vốn không phải là một interface mạng thực tế đang gán trên Pod container.
Kết quả: Spark Master ném lỗi không thể bind địa chỉ

Fix: Cấu hình cho Apache Spark Master trích xuất động địa chỉ IP thực tế trực tiếp từ Pod ngay khi khởi chạy thông qua Downward API gán vào biến hệ thống.

- Vòng lặp vô hạn Kafka Controller bootstrap (Chicken-and-egg loop): Broker Kafka khi khởi động tự động kết nối mạng Service DNS kafka:9092 nội bộ. Tuy nhiên, Service K8s lại từ chối phân phối địa chỉ endpoint do Pod chưa đạt trạng thái Ready.

Kết quả: broker không thể khởi động, tiến trình readiness probe báo fail, sập luồng bootstrap.

Fix: Kích hoạt thuộc tính cờ mạng publishNotReadyAddresses: true trên Service định tuyến của Kafka để ép hệ thống tự mở cổng kết nối mạng động phục vụ cho Controller của Kafka.

- Lỗi treo lệnh Kafka readiness probe (exec probe hang): Sử dụng các script câu lệnh kiểm tra nội bộ dạng kafka-topics --list chạy dạng exec probe bị block cứng vô hạn bên trong cửa sổ thời gian Zookeeper reconnect.

Kết quả: Kubelet kill tiến trình exec liên tục gây hiện tượng flap Pod.

Fix: Thay thế toàn bộ cơ chế liveness/readiness probe của Kafka từ dạng thực thi exec script công kênh sang kiểm tra kết nối mạng thuần tcpSocket trên port 9092 để đạt độ tin cậy tuyệt đối cho stable-state.

- Lỗi luồng Streaming im lặng không xóa Redis cache: Hàm xử lý dọn dẹp cache invalidate_chat_cache() bị ngắt mạch ngắn im lặng (silently short-circuit) do tệp manifest deployment dành riêng cho luồng streaming (spark-job) bị thiếu sót khai báo biến môi trường chỉ thị đích kết nối mạng REDIS_URL

Kết quả: Hệ thống hoàn toàn không sinh ra error log cụ thể, nhưng chatbot bị hiện tượng cache stale kéo dài suốt 1 tiếng đồng hồ bất kể văn bản mới đã nạp thành công vào Elasticsearch.

Fix: Khai báo bổ sung chính xác biến môi trường kết nối mạng REDIS_URL vào env của Deployment spark-job thời gian thực.

- Áp dụng các mẫu thiết kế giảm thiểu cascade: Hệ thống thử nghiệm cơ chế không cô lập lỗi, chấp nhận partial hoặc graceful degradation khi gặp sự cố mạng ngoại vi.

Điều rút ra: Việc đưa hệ thống Big Data phân tán phức tạp lên vận hành trên cụm Kubernetes không phải là một tác vụ drop-in đơn giản do nhiều hành vi cấu hình mặc định của K8s rất dễ xung đột và phá vỡ cấu trúc container của ứng dụng. Vì vậy, các cờ thuộc tính cô lập như **enableServiceLinks: false** nên được thiết lập làm quy chuẩn mặc định cho mọi workload chạy trong vùng namespace có đa dịch vụ, kết hợp cờ mạng **publishNotReadyAddresses: true** bắt buộc cho các cấu phần tự nhận diện kết nối lúc bootstrap như Kafka. Thay vì dùng exec probe cổng kênh, tác vụ probe mạng qua cổng tcpSocket mang lại độ tin cậy cao hơn cho trạng thái stable-state, đồng thời các bug ẩn dạng im lặng do thiếu biến môi trường buộc hệ thống phải duy trì quy trình validate khép kín đầu-cuối ngay sau khi triển khai và đảm bảo các luồng quản trị như audit log không bao giờ nằm trên luồng chặn của người dùng.

4.6. Tối ưu hiệu năng

Mô tả vấn đề:

- **Bối cảnh và nền tảng:** Hệ thống triển khai giao diện Hỏi-Đáp pháp luật sử dụng mô hình RAG (Retrieval-Augmented Generation) kết hợp mô hình ngôn ngữ lớn GPT-4o mini và mô hình tạo vector nhúng text-embedding-3-small để phục vụ người dùng tra cứu văn bản quy phạm.
- **Thách thức gặp phải:** Hệ thống cần đáp ứng nghiêm ngặt các tiêu chuẩn về mặt hiệu năng bao gồm độ trễ phản hồi của chatbot (Latency) phải kiểm soát dưới ngưỡng 10 giây (bao gồm cả thời gian sinh từ GPT) và độ trễ của riêng tầng tìm kiếm (Search latency) phải đạt mức dưới 200ms. Đồng thời, chi phí tài chính gọi các dịch vụ API của OpenAI cần phải được kiểm soát tối ưu.
- **Tác động đến hệ thống:** Nếu không có cơ chế tối ưu, mỗi lượt tương tác của người dùng đều bắt buộc phải gọi API ngoại vi để sinh vector câu hỏi

và sinh từ LLM, gây tốn kém chi phí Token, làm chậm tốc độ phản hồi tổng thể và làm giảm trải nghiệm người dùng cuối.

Giải pháp cuối cùng: Hệ thống triển khai đồng bộ giải pháp Bộ nhớ đệm Redis nâng cao kết hợp Tinh chỉnh Tìm kiếm lai phân tầng.

- Triển khai chi tiết:
 - Sử dụng bộ mã hóa khóa băm SHA-256 dựa trên nội dung câu hỏi ("chat:" + sha256(cau_hoi)), thiết lập thời gian sống dữ liệu TTL 3600 giây. Tích hợp cơ chế xóa bộ đệm chủ động kích hoạt tự động theo sự kiện ghi thành công từ Spark (event-driven invalidation) để dọn sạch các khóa liên quan ngay sau khi hệ thống kết thúc lượt chạy lô (batch run).
 - Chuyển đổi toàn bộ sang kỹ thuật driver-side batch embedding với cấu hình đóng gói gom cụm 100 đoạn văn bản trong một lượt gọi duy nhất.
 - Xây dựng cơ sở dữ liệu chỉ mục lai dựa trên không gian lập chỉ mục dense_vector 1536 chiều bằng cấu hình Elasticsearch HNSW index, tính toán khoảng cách Cosine. Thiết lập bộ phân tách từ custom Vietnamese ICU analyzer với icu_tokenizer và filter chain icu_normalizer → icu_folding → lowercase để đồng bộ Unicode NFC cho tiếng Việt.
 - Cấu hình bộ số hiệu chuẩn cân bằng trọng số (boost weights) chi tiết trong FastAPI backend: gán boost = 0.3 cho tìm kiếm ngữ nghĩa vector knn, gán boost = 5.0 cho trường văn bản đoạn chunk_text, và gán boost = 3.0 cho trường tiêu đề title, thực hiện thu giữ kết quả ở ngưỡng top_k = 10.
- Chỉ số và kết quả: Cache Hit đạt mức tối ưu dưới 500ms; Cache Miss dao động trong khoảng từ 5 giây đến 8 giây.

Điểm rút ra: Tỷ lệ trùng bộ nhớ đệm (Cache hit rate) là số liệu cấu thành quan trọng nhất để tối ưu hóa chi phí vận hành, vì mỗi lượt cache miss sẽ bắt buộc hệ thống phải tiêu tốn thêm một lượt gọi OpenAI call trả phí. Việc triển khai cơ chế xóa cache chủ động dựa trên sự kiện mang lại hiệu quả vượt trội so với cấu hình thời gian sống thụ động, giúp

người dùng không bao giờ phải tiếp cận thông tin cũ lỗi thời ngay sau khi lớp Batch layer hoàn tất chu kỳ xử lý lại. Cuối cùng, do quá trình tinh chỉnh trọng số tìm kiếm lai (Hybrid search tuning) mang bản chất thực nghiệm sâu sắc (empirical), quản trị viên bắt buộc phải xây dựng và dựa vào một hệ thống Framework đánh giá chất lượng (eval framework) để thực hiện kiểm thử và lựa chọn ra bộ trọng số (weights) tối ưu nhất.

4.7. Giám sát và gỡ lỗi

Mô tả vấn đề:

- **Bối cảnh và nền tảng:** Hệ thống phân tán bao gồm hơn 10 thành phần dịch vụ dịch vụ vận hành đồng thời trong một hạ tầng K8s namespace duy nhất, đi kèm luồng dữ liệu streaming xử lý thời gian thực hoạt động liên tục 24/7 của Apache Spark.
- **Thách thức gặp phải:** Quản trị viên cần phải theo dõi, giám sát toàn diện sức khỏe phần cứng, đồng thời bắt buộc phải đo đạc, theo dõi chi phí tài chính gọi dịch vụ AI chi tiết theo từng lượt yêu cầu
- **Tác động đến hệ thống:** Thiếu hụt một cơ chế thu thập chỉ số tập trung sẽ làm hệ thống mất đi tính minh bạch, không thể phát hiện các điểm nghẽn hạ tầng của luồng streaming và không thể hạch toán chính xác hóa đơn Token của chatbot.

Giải pháp cuối cùng: Hệ thống thiết lập kiến trúc Giám sát phân tầng tách biệt (Prometheus, Grafana) và Phân tích nghiệp vụ hội tụ (Kibana, Audit log).

- Triển khai chi tiết:
 - Hạ tầng giám sát kỹ thuật (Prometheus + Grafana): Cấu hình Prometheus cấp Cluster-level RBAC, áp dụng tính năng tự động phát hiện dịch vụ quét metrics dựa trên thuộc tính annotation gắn tại tệp manifest: `prometheus.io/scrape: "true"`. Grafana được cấu hình nạp sẵn Datasource tự động, thiết lập chu kỳ lưu trữ cố định trong vòng 15 ngày.

- Hạ tầng phân tích nghiệp vụ (Kibana + Audit Log): Thiết lập tệp cấu hình nạp Dashboard chuyên dụng mang tên Legal Chatbot - Document Analytics trên Kibana. Toàn bộ dữ liệu log kiểm toán đi qua FastAPI backend được đẩy vào Elasticsearch audit index, kết xuất trực quan lên Kibana Dashboard theo dõi thời gian thực 5 chỉ số vận hành lõi bao gồm: lưu lượng truy vấn theo khung giờ (query volume by hour), cơ cấu danh mục luật phổ biến (top categories), tỷ lệ trúng bộ nhớ đệm (cache hit rate), biểu đồ phân bố độ trễ xử lý (latency P95), và thống kê số lượng thông tin nhạy cảm bị ngăn chặn (PII redaction count).
- Hạ tầng quản trị hỗ trợ: Toàn bộ luồng Log nghiệp vụ của Apache Spark được cấu hình kết xuất trực tiếp ra luồng đầu ra tiêu chuẩn (stdout) để phục vụ truy vết nhanh qua lệnh kubectl logs. Sử dụng các câu lệnh truy vấn đếm siêu nhẹ _count API của Elasticsearch để liên tục giám sát và track kích thước quy mô của kho dữ liệu văn bản pháp luật (corpus size).
- Chỉ số và kết quả: Toàn bộ hệ thống giám sát khởi chạy trơn tru ngay sau lệnh deploy cluster mà không cần bất kỳ thao tác nhấp chuột thủ công nào. Các panel chỉ số của Grafana hiển thị chính xác độ trễ và các hành vi tương tác của luồng xử lý RAG.

Điều rút ra: Dữ liệu log kiểm toán đóng vai trò là một Dashboard trung tâm phản ánh đồng thời cả hai trạng thái: system health và user behavior. Việc phân tích chi phí vận hành bao gồm số lượng LLM tokens tiêu hao hay số lượt gọi embedding calls bắt buộc phải được thiết kế lưu vết dưới dạng dữ liệu có cấu trúc ngay từ giai đoạn đầu xây dựng mã nguồn. Cuối cùng, một thực hành quản trị tốt cho hệ thống Big Data là phân tách rạch ròi vai trò của công cụ: sử dụng Kibana chuyên biệt cho việc phân tích dữ liệu nghiệp vụ (data analytics) và sử dụng Grafana tập trung cho việc giám sát hạ tầng kỹ thuật (infra metrics).

4.8. Mở rộng

Mô tả vấn đề:

- **Bối cảnh và nền tảng:** Hệ thống được vận hành và triển khai thực tế trên cấu hình phần cứng thiết bị máy vật lý ram 16gb sử dụng môi trường Kubernetes cục bộ.
- **Thách thức gặp phải:** Tài nguyên bộ nhớ RAM vật lý bị giới hạn nghiêm ngặt ở mức 16GB. Trong khi đó, hệ thống ở trạng thái Production thực tế có thể xuất hiện yêu cầu bắt buộc phải xử lý và gánh vác tải lượng khối lượng dữ liệu lớn tăng trưởng gấp 10 lần
- **Tác động đến hệ thống:** ầu hình cấp phát tài nguyên không hợp lý sẽ dẫn đến nghẽn thắt nút cổ chai hoặc gây sập toàn bộ cluster do cạn kiệt bộ nhớ vật lý của máy host.

Giải pháp cuối cùng: Hệ thống thiết lập giải pháp Phân rã Workload kết hợp Ràng buộc tài nguyên nghiêm ngặt ở quy mô Minikube.

- Triển khai chi tiết:
 - Cấu hình Apache Spark: Phân tách rạch ròi thành phần Spark Master và Spark Worker sang các tệp cấu hình Deployment biệt lập, cho phép linh hoạt co giãn số lượng workers khi cần. Chỉ định dung lượng bộ nhớ RAM cố định cho mỗi Spark Worker ở mức 2GB.
 - Cấu hình Elasticsearch & Redis: Triển khai Elasticsearch ở mô hình Single-node kết hợp lập chỉ mục HNSW index tải trực tiếp in-memory để tối ưu hóa tốc độ truy xuất ngữ cảnh. Giới hạn dung lượng bộ nhớ heap của Elasticsearch xuống mức tối đa 512MB. Thiết lập Redis dưới dạng Single-node áp dụng thuật toán giải phóng bộ nhớ đệm LRU eviction khi đầy.
 - Quản trị tài nguyên K8s: Bổ sung nghiêm ngặt các thông số giới hạn và yêu cầu tài nguyên (K8s resource requests/limits per container) vào phần khai báo của tất cả các tệp manifest deployment trong cluster.
- Chỉ số và kết quả: Toàn bộ hệ thống cluster vận hành song song ổn định, tổng RAM tiêu thụ được kiểm soát an toàn dưới ngưỡng 8GB trên máy vật lý.

Điểm rút ra: Để đáp ứng tải trọng dữ liệu lớn ở môi trường Production thực tế, hệ thống cần áp dụng chiến lược co giãn theo chiều ngang đối với các thành phần phi trạng thái như FastAPI replicas, Kafka brokers, và mở rộng thêm các cấu phần Spark workers, kết hợp co giãn theo chiều dọc dung lượng RAM heap phục vụ cho việc lưu trữ HNSW index in-memory của Elasticsearch. Việc lựa chọn giữa mô hình tự chạy cấu phần nhúng (self-host embedding model) hay sử dụng dịch vụ ngoại vi (OpenAI API) mang bản chất đánh đổi về mặt chi phí tính toán phụ thuộc trực tiếp vào tần suất lượng truy vấn hệ thống. Cuối cùng, quản trị viên có thể tối ưu từ 60% đến 80% chi phí hạ tầng tính toán bằng cách cấu hình sử dụng các node giá rẻ (Spot/preemptible compute) riêng cho luồng xử lý lô Batch Layer nhờ đặc tính cho phép ngắt mạch và tự khôi phục trạng thái offset.

4.9. Chất lượng dữ liệu và kiểm thử

Mô tả vấn đề:

- **Bối cảnh và nền tảng:** Hệ thống thực hiện xử lý dữ liệu lớn dạng văn bản phi cấu trúc từ môi trường Web để nạp vào kho chỉ mục tìm kiếm lai phục vụ cho mô hình ngôn ngữ lớn sinh câu trả lời.
- **Thách thức gặp phải:** ăn bản thu thập thô từ Web thường xuyên chứa các đoạn mã rác HTML và bị sai lệch bảng mã ký tự. Thêm vào đó, logic phân tách đoạn văn bản sở hữu độ phức tạp cao, dễ gây ra lỗi regression logic khi sửa đổi mã nguồn. Cuối cùng, chất lượng đầu ra của mô hình RAG mang tính chất cảm tính, chủ quan, rất khó để đo đạc và định lượng bằng các phương pháp thông thường
- **Tác động đến hệ thống:** Sự thiếu hụt của các công cụ đo lường tự động khiến hệ thống không thể kiểm soát chất lượng dữ liệu nạp, dễ làm sai cấu trúc câu tiếng Việt, trực tiếp làm suy giảm độ chính xác của bước truy xuất context ngữ cảnh.

Giải pháp cuối cùng: Hệ thống triển khai đồng bộ giải pháp Tách biệt tầng kiểm thử mã nguồn (Unit Tests) phối hợp Framework đánh giá ngoại tuyến (Offline Evaluation).

- Triển khai chi tiết:

- Refactor mã nguồn phục vụ Testability: Tiến hành bóc tách toàn bộ logic xử lý chuỗi chữ thô ra khỏi wrapper UDF của Spark, đặt độc lập tại tệp mã nguồn Python thuần spark/utis/text.py. Sự phân tách này giúp bộ kiểm thử đơn vị (Unit test) có khả năng thực thi ngay lập tức mà hoàn toàn không cần SparkSession.
- Hạ tầng Unit tests (tests/): Xây dựng bộ kịch bản gồm 16 unit tests thực thi tự động qua thư viện pytest: test_chunker.py chịu trách nhiệm kiểm tra hàm clean_text và hàm chunk_text; test_audit.py chịu trách nhiệm kiểm tra tính chính xác của các biểu thức Regex dùng để quét sạch thông tin PII nhạy cảm
- RAG evaluation framework (eval/): Thiết lập tập dữ liệu chuẩn gold_set.json bao gồm 12 câu truy vấn tiếng Việt mẫu đi kèm danh sách các từ khóa kỳ vọng và danh mục phân loại chuẩn. Script điều khiển run_eval.py được thiết kế linh hoạt chạy song song ở 2 chế độ: backend - chạy toàn trình hệ thống RAG pipeline, có phát sinh LLM cost và direct: Chỉ chạy riêng tầng truy xuất dữ liệu, không LLM. Thống kê đo đạc trực tiếp qua 4 chỉ số : precision@k, recall@k, category hit rate và mean latency.
- Schema validation & Dedup: Áp dụng ràng buộc nghiêm ngặt cấu trúc Kafka schema bên trong file spark/streaming/consumer.py phục vụ luồng xử lý nhanh, phối hợp khai báo RAW_DOC_SCHEMA tại file spark/batch/pipeline.py phục vụ luồng xử lý lô. Tích hợp cơ chế tự động phát hiện trùng lặp dữ liệu lớn thông qua hàm Spark fetch_existing_doc_ids kết hợp thuật toán sinh mã _id deterministic của Elasticsearch.
- Chỉ số và kết quả: Toàn bộ hệ thống kiểm thử unit test tự động thực thi thành công với thời gian chạy siêu ngắn chỉ khoảng 0.1 giây.

Điểm rút ra: Thiết kế mã nguồn sử dụng hàm Python thuần túy mang lại lợi ích vượt trội về mặt kiểm thử (Pure Python > UDF when testability matters) so với việc đặt logic nằm sâu trong các wrapper phụ thuộc của Big Data Framework. Hệ thống đánh giá chất lượng RAG bắt buộc phải sở hữu một tập dữ liệu chuẩn ngoại tuyến (gold-set offline)

để liên tục đo đạc, phát hiện lỗi regression của thuật toán tìm kiếm mà không làm tiêu tốn chi phí tài chính Token LLM call cho mỗi lượt chạy. Cuối cùng, việc thiết lập song song cơ chế kiểm tra cấu trúc dữ liệu khi đọc của Spark kết hợp kiểm tra cấu trúc khi ghi của Elasticsearch tạo nên một hệ thống phòng thủ kép chặt chẽ cho chất lượng dữ liệu lớn (Schema-on-read + schema-on-write = belt + suspenders).

4.10. Bảo mật và quản trị

Mô tả vấn đề:

- **Bối cảnh và nền tảng:** Khi chatbot đi vào vận hành thực tế, hệ thống phải tương tác trực tiếp với các API bên thứ ba (OpenAI) và tiếp nhận hàng loạt câu hỏi từ người dùng cuối.
- **Thách thức gặp phải:** Khóa bảo mật OpenAI API key cần được bảo vệ nghiêm ngặt, tránh rò rỉ dưới dạng text thuần (plaintext) trong mã nguồn hoặc tệp cấu hình manifest. Đồng thời, nội dung câu hỏi nhập vào từ phía người dùng có rủi ro cao chứa các thông tin định danh cá nhân nhạy cảm cần phải được làm sạch trước khi đưa vào hệ thống lưu trữ log kiểm toán nhằm tuân thủ các quy định bảo mật dữ liệu.
- **Tác động đến hệ thống:** Việc để lộ API key sẽ dẫn đến rủi ro bị đánh cắp tài nguyên và leo thang chi phí trực lợi. Việc lưu trữ trực tiếp thông tin PII của người dùng vào log sẽ vi phạm nghiêm trọng quyền riêng tư và các tiêu chuẩn tuân thủ an toàn thông tin.

Giải pháp cuối cùng: Triển khai cơ chế mã hóa tài nguyên tĩnh thông qua cấu trúc mã nguồn K8s kết hợp với module PII Sanitization bất đồng bộ tại tầng Backend.

- Triển khai chi tiết:
 - Mã hóa Credential: Khóa kết nối OPENAI_API_KEY được bao bọc bên trong đối tượng K8s Secrets (backend-secret) và được mount an toàn vào container ứng dụng thông qua cấu hình khai báo mật envFrom: secretRef.

- Module Audit Trail Service: Tại mã nguồn backend, hệ thống xây dựng cấu phần dịch vụ AuditService (backend/services/audit.py). Khi người dùng gửi câu hỏi, module này sử dụng 4 biểu thức chính quy (theo thứ tự EMAIL → PHONE → TAX_ID → CMND/CCCD, trong đó CMND 9 chữ số và CCCD 12 chữ số dùng chung một regex duy nhất `\b\d{9}\b|\b\d{12}\b`) để quét và tiến hành che giấu hoàn toàn dữ liệu PII thành các nhãn định danh chung: [EMAIL], [PHONE], [ID], [TAX_ID].
- Cơ chế ghi log bảo mật: Câu hỏi gốc của người dùng bị hủy bỏ ngay lập tức tại luồng audit log. Hệ thống chỉ tiến hành lưu trữ chuỗi mã hóa băm một chiều SHA-256 kết hợp cùng chuỗi nội dung sạch đã qua xử lý question_sanitized sang chỉ mục phapluat-audit của Elasticsearch bằng cơ chế gọi bất đồng bộ không chặn (best-effort pattern) nhằm không gây ảnh hưởng đến luồng trả lời chính.
- Chỉ số và kết quả: Log kiểm toán được ghi nhận sạch hoàn toàn dấu vết PII và hiển thị trực quan an toàn trên hệ thống quản trị. Toàn bộ các API key được bảo vệ nghiêm ngặt và vượt qua các bài kiểm tra rà soát mã nguồn.

Điểm rút ra: Công tác làm sạch dữ liệu nhạy cảm phải được thực thi tại layer thấp nhất ngay khi tiếp nhận dữ liệu đầu vào. Việc lưu trữ kết hợp chuỗi Hash một chiều và văn bản đã qua tinh lọc là giải pháp tối ưu giúp hệ thống vừa duy trì năng lực dò vết, gỡ lỗi vừa đảm bảo tuân thủ tuyệt đối quyền riêng tư. Ngoài ra, việc xoay vòng khóa cần được tự động hóa thông qua cơ chế restart rollout deployment của Kubernetes.

4.11. Chịu lỗi

Mô tả vấn đề:

- **Bối cảnh và nền tảng:** Cơ sở dữ liệu Elasticsearch đóng vai trò là hạt nhân lưu trữ toàn bộ vector embedding và các phân đoạn văn bản phục vụ cho tầng Serving layer.

- **Thách thức gặp phải:** Hệ thống phải đối mặt với nguy cơ dữ liệu chỉ mục của Elasticsearch bị hỏng hóc vật lý hoặc corrupt hệ tệp tin do lỗi mất điện đột ngột hoặc sập Pod container. Thêm vào đó, phiên bản Elasticsearch 8.x áp dụng các quy tắc an ninh hardening cực kỳ nghiêm ngặt: cấm hoàn toàn việc khai báo thông số bảo mật credentials inline và loại bỏ cờ allow_insecure_settings qua biến môi trường, gây lỗi sập tiến trình unknown setting boot error khi khởi động.
- **Tác động đến hệ thống:** Xung đột cấu hình bảo mật của ES 8.x khiến cụm dịch vụ không thể khởi động.

Cách tiếp cận đã thử:

- **Cách 1:** Cách 1: Khai báo trực tiếp access_key và secret_key inline trong thân câu lệnh (body) khi gọi API đăng ký repository (chủ cũ của ES 6/7).
Kết quả: Bản Elasticsearch 8.12.2 từ chối nhận lệnh và trả về lỗi an ninh: "Setting [access_key] is insecure, but property [allow_insecure_settings] is not set".
- **Cách 2:** Kích hoạt thuộc tính cấu hình mở cổng escape hatch repositories.s3.allow_insecure_settings=true thông qua biến môi trường của container.
Kết quả: Elasticsearch 8.12.2 bị sập tiến trình khởi động (boot fail) với thông báo lỗi "unknown setting" do thuộc tính này đã bị xóa bỏ hoàn toàn khỏi lõi 8.x

Giải pháp cuối cùng: Ứng dụng mẫu thiết kế ES keystore initContainer pattern nhằm vượt qua rào cản an ninh của ES 8.x, phối hợp cùng quy trình thiết lập chiến lược sao lưu ảnh đĩa tự động hóa.

- Triển khai chi tiết:
 - Cấu hình ES Snapshot → MinIO: Sử dụng module repository-s3 được tích hợp sẵn (bundled) trong lõi Elasticsearch 8.12.2 mà không cần cài đặt thêm plugin bên ngoài. Tạo một đối tượng Kubernetes Secret mang

tên minio-creds để bao bọc access_key và secret_key. Thiết lập một cấu phần khởi tạo load-keystore (initContainer) chạy chính image của ES để thực thi lệnh nội bộ elasticsearch-keystore create → nạp các khóa cấu hình bảo mật hệ thống vào tệp keystore an toàn từ biến môi trường → sao chép file keystore này vào một ổ đĩa tạm thời chung emptyDir. Container Elasticsearch chính khi boot sẽ thực hiện mount file keystore này thông qua đường dẫn con subPath ở chế độ chỉ đọc (read-only).

- Chiến lược Triển khai Hạ tầng: Ép cấu hình tệp manifest Deployment của Elasticsearch sang chiến lược Recreate nhằm triệt tiêu hoàn toàn lỗi xung đột khóa tệp tin node.lock phát sinh do cuộc đua rolling update (rolling update race) của đĩa PVC gán thuộc tính ReadWriteOnce. Thiết lập K8s CronJob es-snapshot vận hành tự động để kích hoạt script scripts/snapshot_elasticsearch.py snapshot, thực hiện lưu vết chỉ mục sang bucket es-snapshots/ trên MinIO. Lệnh phục hồi dữ liệu được kích hoạt linh hoạt qua API POST_snapshot/phapluat-snapshots/snap-XXX/_restore.
- Cơ chế Chịu lỗi luồng Streaming: Kích hoạt thuộc tính lưu vết mốc phân vùng Kafka offset checkpoint tại hệ thống tệp tin cục bộ checkpoints/streaming_python/. Toàn bộ khối văn bản lập chỉ mục sử dụng mã định danh deterministic chunk_id cố định dựa trên nội dung băm nhằm biến mọi tác vụ nạp lại thành hành vi ghi đè (upsert an toàn), cho phép tự động khôi phục và chạy lại mà không làm trùng lặp dữ liệu lớn.
- Hạ tầng Phòng thủ Dữ liệu phụ: Kho dữ liệu bất biến Master Dataset dạng JSON lưu tại phapluat/master/ hỗ trợ khả năng phát lại toàn diện (replay toàn corpus) khi kích hoạt cờ lực lượng BATCH_FORCE=true. Hệ thống duy trì thêm tệp CSV dự phòng (phapluat/csv/backup/) do crawler kết xuất để làm lớp bảo vệ cuối cùng.

- Chỉ số và kết quả: Xác thực toàn diện: Hệ thống ghi nhận ảnh chụp ảnh đĩa snapshot đạt trạng thái SUCCESS hoàn toàn. Quá trình đóng gói tự động nén toàn bộ tri thức của cả hai index [phapluat, phapluat-audit] thành 28 tệp nhị phân dữ liệu (tổng dung lượng ~5MB) lưu trữ an toàn, phục hồi nguyên vẹn trạng thái hệ thống khi chạy thử nghiệm.
- Hạn chế: Cấu hình hạ tầng trong phạm vi demo đang chạy ở dạng Single-node (Kafka replication factor = 1, MinIO single-node không erasure-coding). Ở môi trường Production thực tế, hệ thống bắt buộc phải nâng cấp lên cụm Kafka tối thiểu ≥ 3 brokers và kích hoạt kiến trúc MinIO cluster mode phân tán.

Điểm rút ra: Cơ chế bảo mật thắt chặt an ninh của Elasticsearch 8.x loại bỏ hoàn toàn việc truyền inline credentials, do đó kiến trúc sử dụng Keystore bảo mật phối hợp mô hình initContainer + emptyDir + subPath mount trên Kubernetes là giải pháp chuẩn Production duy nhất có thể áp dụng. Chiến lược bảo vệ dữ liệu lớn cần được phân tầng chặt chẽ thành Mô hình phòng thủ 3 lớp (3-tier backup strategy) bao gồm: ES snapshot (khôi phục trạng thái nhanh tại chỗ - fast restore), MinIO master/ JSON (phát lại toàn diện khi re-process), và CSV backup thô (tuyến phòng thủ cuối cùng - last resort). Tính chất đĩa PVC gán thuộc tính ReadWriteOnce buộc quản trị viên phải thiết lập chiến lược Deployment dạng Recreate cho các stateful workload để tránh lỗi tranh chấp file lock. Cuối cùng, việc áp dụng thiết kế bất biến dựa trên mã ID xác thực giúp chuyên hóa toàn bộ quy trình "khắc phục sự cố phức tạp" thành một tác vụ "chạy lại lệnh" (rerun) tự động, hoàn toàn giải phóng sức lao động thủ công của con người.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết quả đạt được

Dự án đã hoàn thành thiết kế, xây dựng và cấu hình thực tế hệ thống Hỏi-Đáp pháp luật đầu-cuối (end-to-end), vượt qua toàn bộ các bài kiểm tra runtime và được validate thành công trên môi trường Kubernetes cục bộ (Minikube Target). Các kết quả định lượng cụ thể bao gồm:

- Về quy mô nạp dữ liệu của Batch Layer: Hệ thống đã tiến hành thu thập, xử lý và nạp thành công 2.070 văn bản pháp luật thực tế trải dài trên 33 danh mục phân loại nghiệp vụ khác nhau. Toàn bộ dữ liệu được băm nhỏ và lập chỉ mục không gian vector với quy mô ~11.160 khối đoạn văn (chunks) lưu trữ trong Elasticsearch. Quá trình bóc tách đạt tỷ lệ 89% số văn bản trích xuất chính xác thuộc tính ngày đăng văn bản \ trực tiếp từ cấu trúc mã JSON-LD của trang web nguồn.
- Về cơ chế tối ưu chi phí Batch Layer: Chế độ mặc định BATCH_FORCE=false (dedup-skip mode) thông qua hàm fetch_existing_doc_ids giúp hệ thống đối chiếu aggregation từ Elasticsearch để chủ động bỏ qua các văn bản đã có. Cơ chế này giúp tiết kiệm tới 99% chi phí gọi API OpenAI cost sau lượt nạp tri thức ban đầu.

- Về hiệu năng thời gian thực của Speed Layer: Thời gian hoàn thành chu kỳ đầu-cuối đo đặc thực tế dao động trong khoảng ~45 đến 50 giây (bao gồm cả độ trễ bóc tách thực tế từ máy chủ của website nguồn).
- Về chất lượng Hỏi - Đáp tầng Serving: Mô hình tìm kiếm lai Hybrid Search phối hợp kNN HNSW vector search (không gian 1536 chiều, khoảng cách Cosine) và full-text BM25 thông qua ICU Vietnamese Analyzer đã vận hành ổn định trên một chỉ mục pháp luật hội tụ duy nhất. Chatbot trả về các câu trả lời có tính xác thực cao, hiển thị tường minh mã định danh doc_id và URL nguồn trích dẫn gốc từ cấu trúc mảng used_sources do GPT-4o mini phân loại.
- Về công tác Quản trị, Bảo mật và Chịu lỗi: Hệ thống ẩn danh hóa thông tin nhạy cảm hoạt động chính xác. Tác vụ sao lưu an toàn ảnh đĩa định kỳ hàng ngày kích hoạt thành công qua CronJob vào lúc 2 giờ sáng.
- Về khía cạnh Kiểm thử và Giám sát: Hệ thống hoàn thiện bộ 16 kịch bản unit tests tự động chạy thành công qua pytest với tốc độ thực thi siêu ngắn chỉ ~0.1 giây nhờ giải pháp refactor tách biệt hàm Python thuần túy khỏi Spark wrapper. Framework đánh giá ngoại tuyến sẵn sàng đo đặc chất lượng qua chế độ truy xuất trực tiếp không tốn chi phí LLM. Hạ tầng Grafana và Prometheus vận hành ổn định, tự động nạp dashboard nghiệp vụ thông qua tệp json cấu hình sẵn.

5.2. Hạn chế

Mặc dù đạt được các kết quả ấn tượng trên môi trường giả lập thử nghiệm, hệ thống vẫn tồn tại một số điểm giới hạn đặc thù cần được nhìn nhận rõ:

- Cấu hình hạ tầng Single-node: Toàn bộ cụm cluster đang vận hành ở mô hình nút đơn lẻ do giới hạn tài nguyên phần cứng máy vật lý. Hệ thống chứa các điểm nghẽn đơn lẻ (single-point-of-failure) như Apache Kafka chạy single-broker với hệ số sao lưu replication factor = 1, cơ sở dữ liệu Elasticsearch single-node không cluster, và hệ thống MinIO single-node hoàn toàn không cấu hình phân tán xóa mã lỗi
- Thiếu hụt cấu hình Watermark tầng Streaming: Tầng xử lý Structured Streaming thời gian thực hiện tại chưa được khai báo cấu hình từ khóa withWatermark cũng như thiết lập cửa sổ xử lý dữ liệu đến muộn do tập dữ liệu trong scope hiện tại chưa bộc lộ các xáo trộn quá lớn về mốc sự kiện thời gian.
- Các cấu phần phân tích nâng cao chưa được triển khai thực tế: Các thư mục kịch bản và tệp tin stub phục vụ cho việc trình diễn các kỹ năng Spark nâng cao theo yêu cầu lý thuyết hiện tại mới dừng lại ở mức khởi tạo file xương.

5.3. Hướng phát triển

Để nâng cấp hệ thống đạt các tiêu chuẩn sẵn sàng vận hành cho môi trường Production thực tế quy mô lớn, các hướng phát triển tiếp theo của đề án tập trung vào các giải pháp sau:

- Tích hợp cơ chế Versioned Embeddings: Bổ sung thêm trường dữ liệu định danh phiên bản mô hình nhúng vào cấu trúc index của Elasticsearch. Giải pháp này cho phép hệ thống thực hiện nâng cấp mượt mà từng phần tri thức vector mà không bắt buộc phải kích hoạt lệnh xóa toàn bộ và re-embed lại từ đầu toàn corpus gây tốn kém ngân sách.
- Xây dựng bộ lọc tăng trưởng Incremental Watermark Batch: Triển khai cơ chế theo dõi và ghi nhận mốc thời gian của lượt chạy lô gần nhất.
- Mở rộng mạng lưới Crawler đa nguồn: Phát triển và tích hợp bổ sung các module bóc tách mã nguồn chuyên biệt hướng tới thu thập dữ liệu lớn từ các cổng thông tin chính thống khác về pháp luật Việt Nam.
- Tự chủ hạ tầng Embedding: Nghiên cứu thực hiện tinh chỉnh các mô hình nhúng mã nguồn mở chuyên sâu cho ngôn ngữ tiếng Việt pháp lý hoặc tiến hành tự vận hành trực tiếp cụm dịch vụ sinh vector nội bộ trên một phân vùng hạ tầng độc lập của cụm cluster.
- Nâng cấp hạ tầng chịu lỗi ngoại vi: Triển khai quy trình tự động đẩy các tệp tin sao lưu dữ liệu Elasticsearch Snapshot lên các hạ tầng lưu trữ đám mây công cộng độc lập hoàn toàn với Cluster máy vật lý.

DANH MỤC TÀI LIỆU THAM KHẢO

1. **Trần Việt Trung.** *Bài giảng Lưu trữ và xử lý dữ liệu lớn*. Trường Công nghệ Thông tin và Truyền thông – Đại học Bách khoa Hà Nội
2. **Thư Viện Pháp Luật.** *Cơ sở dữ liệu văn bản pháp luật Việt Nam*. Truy cập từ: <https://thuvienphapluat.vn>
3. **Patchright Project.** *Patchright: A Playwright drop-in with stealth patches to bypass anti-bot mechanisms*.
4. **Richardson, L. .** *Beautiful Soup Documentation: Python library for pulling data out of HTML and XML files*. Truy cập từ: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>.
5. **Cloudflare.** *Understanding the Bot Management and Anti-Bot Challenges*. Cloudflare Technical Blog.
6. **OpenAI.** *OpenAI API Documentation: text-embedding-3-small and GPT-4o mini models for RAG*.

7. **Elasticsearch B.V.** . *Elasticsearch Guide (8.x): Approximate nearest neighbor (kNN) search and HNSW algorithm*. Truy cập từ:
<https://www.elastic.co/guide/en/elasticsearch/reference/current/knn-search.html>.
8. **Boto3 SDK**. *Boto3 Documentation: AWS SDK for Python to interact with S3-compatible storage (MinIO)*
9. **The Kubernetes Authors**. *Kubernetes Documentation: Init Containers for secret management and Keystore handling*. Truy cập từ:
<https://kubernetes.io/docs/concepts/workloads/pods/init-containers/>